```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.preprocessing import LabelEncoder
from sklearn.impute import SimpleImputer
import warnings
warnings.filterwarnings('ignore')

df = pd.read_csv('Global_Superstore2.csv', encoding='latin1')
```

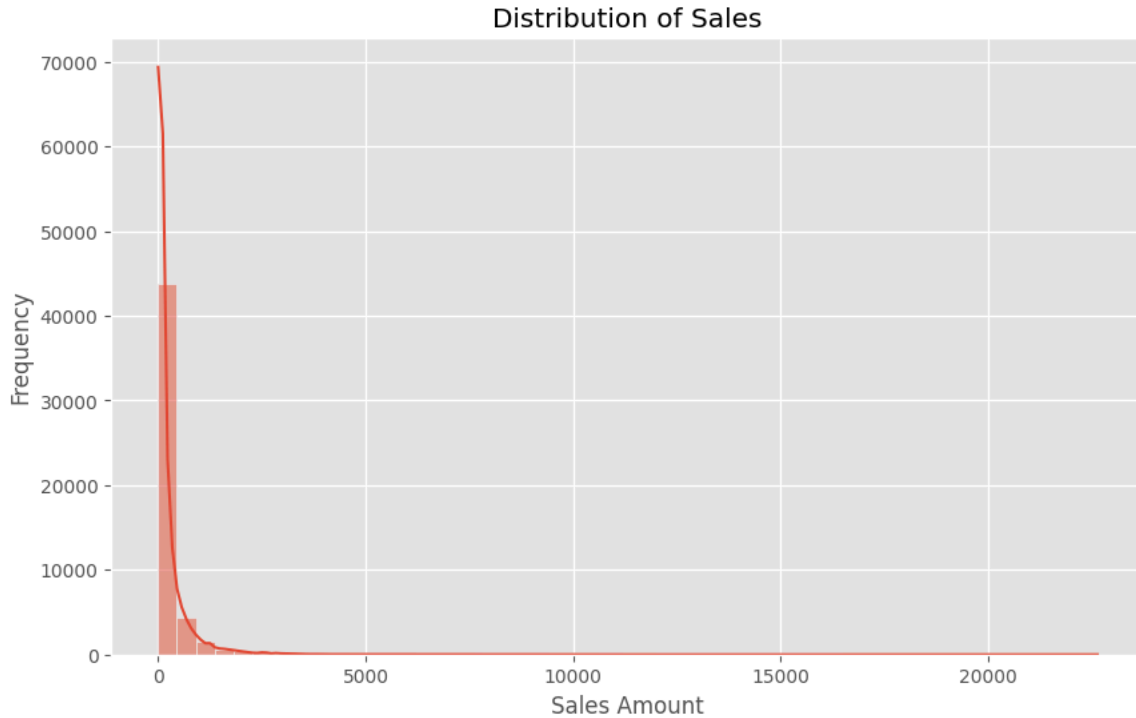
```
print("Dataset Info:")
print(df.info())

print("\nMissing Values:")
print(df.isnull().sum())

df['Postal Code'].fillna('Unknown', inplace=True)
df.dropna(subset=['Customer ID', 'Product ID'], inplace=True)

df['Order Date'] = pd.to_datetime(df['Order Date'], format='%d-%m-%Y')
df['Ship Date'] = pd.to_datetime(df['Ship Date'], format='%d-%m-%Y')

df['Processing Time'] = (df['Ship Date'] - df['Order Date']).dt.days

df['Order Year'] = df['Order Date'].dt.year
df['Order Month'] = df['Order Date'].dt.month
df['Order Day'] = df['Order Date'].dt.day
df['Order Weekday'] = df['Order Date'].dt.weekday

print("\nAfter Cleaning:")
print(f"Shape: {df.shape}")
print("\nMissing Values:")
print(df.isnull().sum().sum())
```

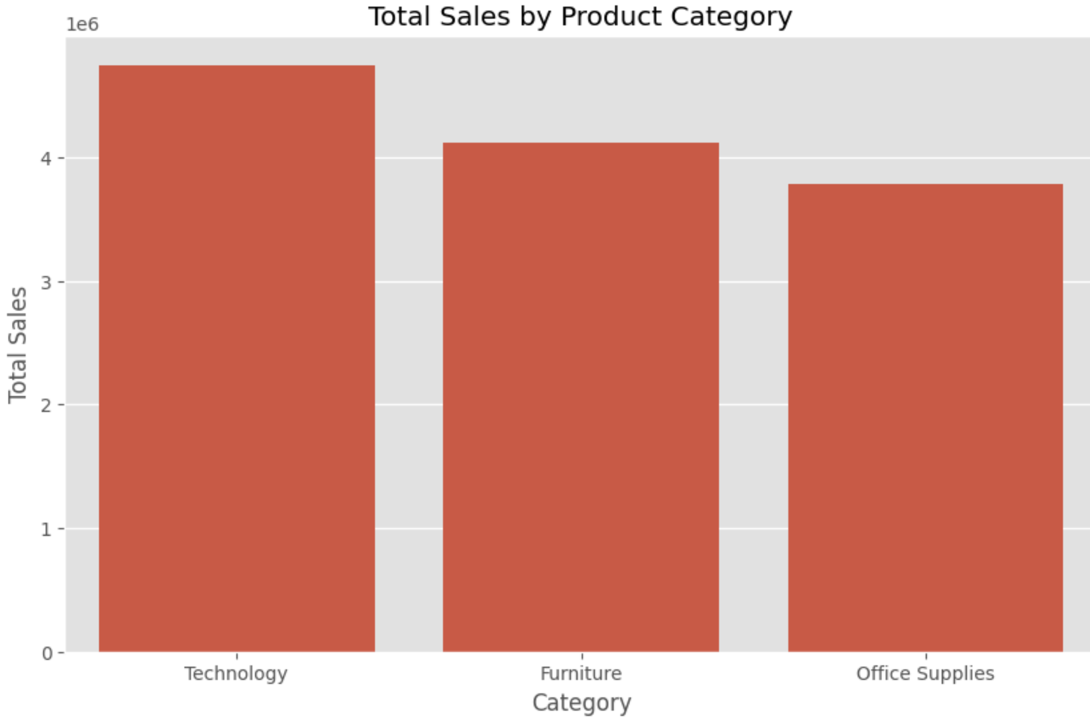
```
 2   Order Date      51290 non-null  object
 3   Ship Date       51290 non-null  object
 4   Ship Mode       51290 non-null  object
 5   Customer ID     51290 non-null  object
 6   Customer Name   51290 non-null  object
 7   Segment         51290 non-null  object
 8   City            51290 non-null  object
 9   State           51290 non-null  object
 10  Country         51290 non-null  object
 11  Postal Code     9994 non-null   float64
 12  Market          51290 non-null  object
 13  Region          51290 non-null  object
 14  Product ID      51290 non-null  object
 15  Category        51290 non-null  object
 16  Sub-Category    51290 non-null  object
 17  Product Name    51290 non-null  object
 18  Sales           51290 non-null  float64
 19  Quantity        51290 non-null  int64
 20  Discount        51290 non-null  float64
 21  Profit          51290 non-null  float64
 22  Shipping Cost   51290 non-null  float64
 23  Order Priority  51290 non-null  object
dtypes: float64(5), int64(2), object(17)
memory usage: 9.4+ MB
None

Missing Values:
Row ID               0
Order ID             0
Order Date           0
Ship Date            0
Ship Mode            0
```

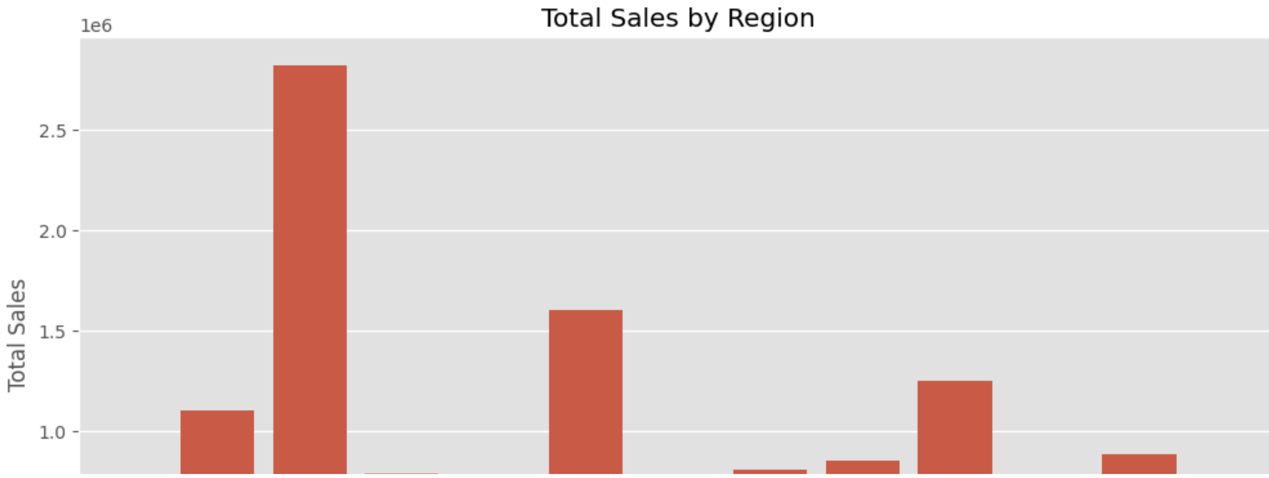
```
Region               0
```

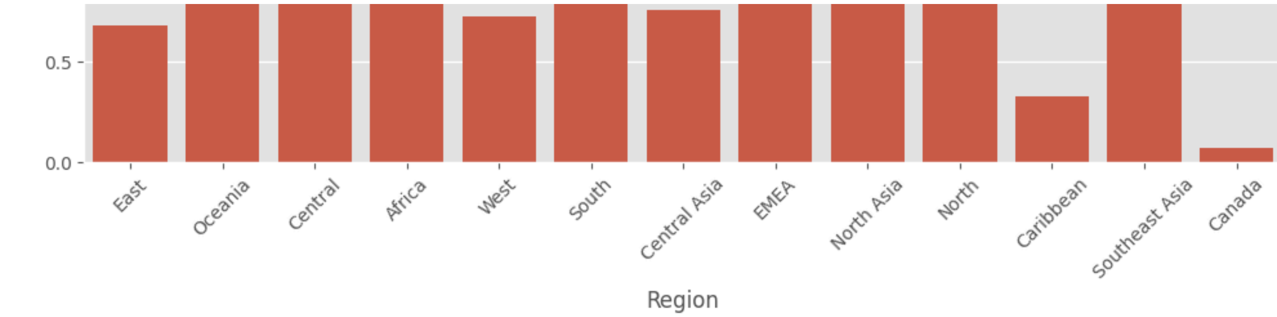
```
Region              0
Product ID          0
Category            0
Sub-Category        0
Product Name        0
Sales               0
Quantity            0
Discount            0
Profit              0
Shipping Cost       0
Order Priority      0
dtype: int64

After Cleaning:
Shape: (51290, 29)

Missing Values:
0
```

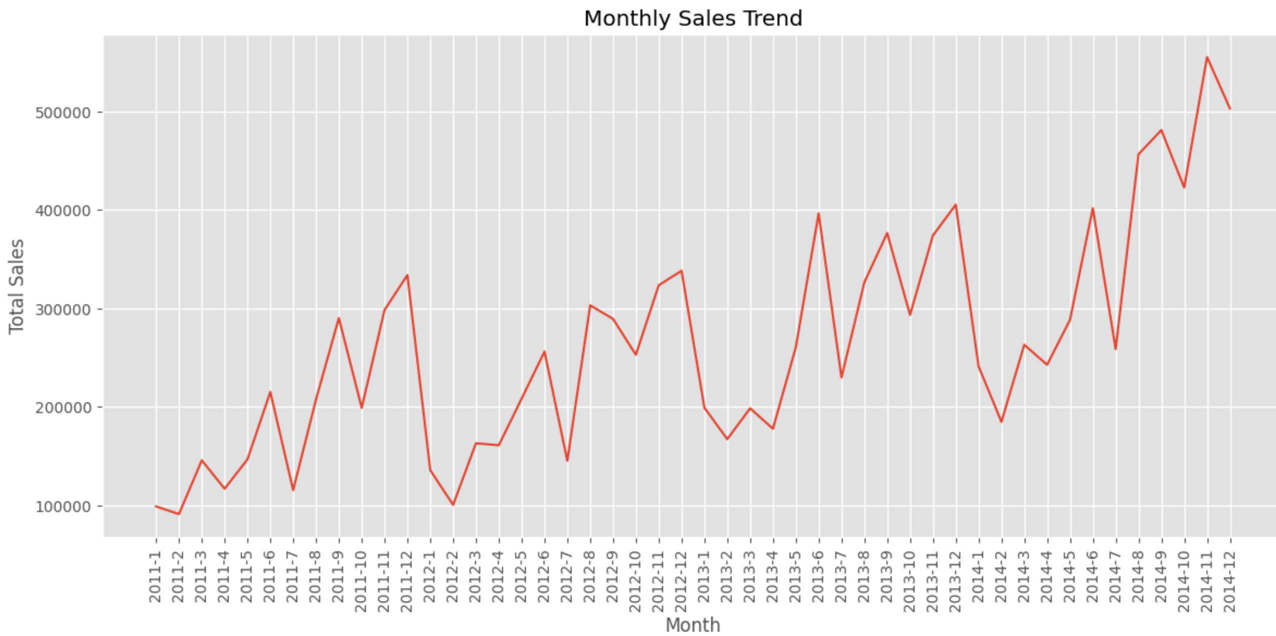
```
plt.style.use('ggplot')

plt.figure(figsize=(10, 6))
sns.histplot(df['Sales'], bins=50, kde=True)
plt.title('Distribution of Sales')
plt.xlabel('Sales Amount')
plt.ylabel('Frequency')
plt.show()

plt.figure(figsize=(10, 6))
sns.barplot(x='Category', y='Sales', data=df, estimator=sum, ci=None)
plt.title('Total Sales by Product Category')
plt.ylabel('Total Sales')
plt.show()

plt.figure(figsize=(12, 6))
sns.barplot(x='Region', y='Sales', data=df, estimator=sum, ci=None)
plt.title('Total Sales by Region')
plt.xticks(rotation=45)
plt.ylabel('Total Sales')
plt.show()

monthly_sales = df.groupby(['Order Year', 'Order Month'])['Sales'].sum().reset_index()
monthly_sales['Year-Month'] = monthly_sales['Order Year'].astype(str) + '-' + monthly_sales['Order Month'].astype(str)

plt.figure(figsize=(14, 6))
sns.lineplot(x='Year-Month', y='Sales', data=monthly_sales)
plt.title('Monthly Sales Trend')
plt.xticks(rotation=90)
plt.xlabel('Month')
plt.ylabel('Total Sales')
plt.show()

plt.figure(figsize=(10, 6))
sns.scatterplot(x='Sales', y='Profit', data=df, alpha=0.5)
plt.title('Profit vs. Sales')
plt.show()

numeric_cols = df.select_dtypes(include=[np.number]).columns
plt.figure(figsize=(12, 8))
sns.heatmap(df[numeric_cols].corr(), annot=True, cmap='coolwarm', center=0)
plt.title('Correlation Heatmap')
plt.show()
```

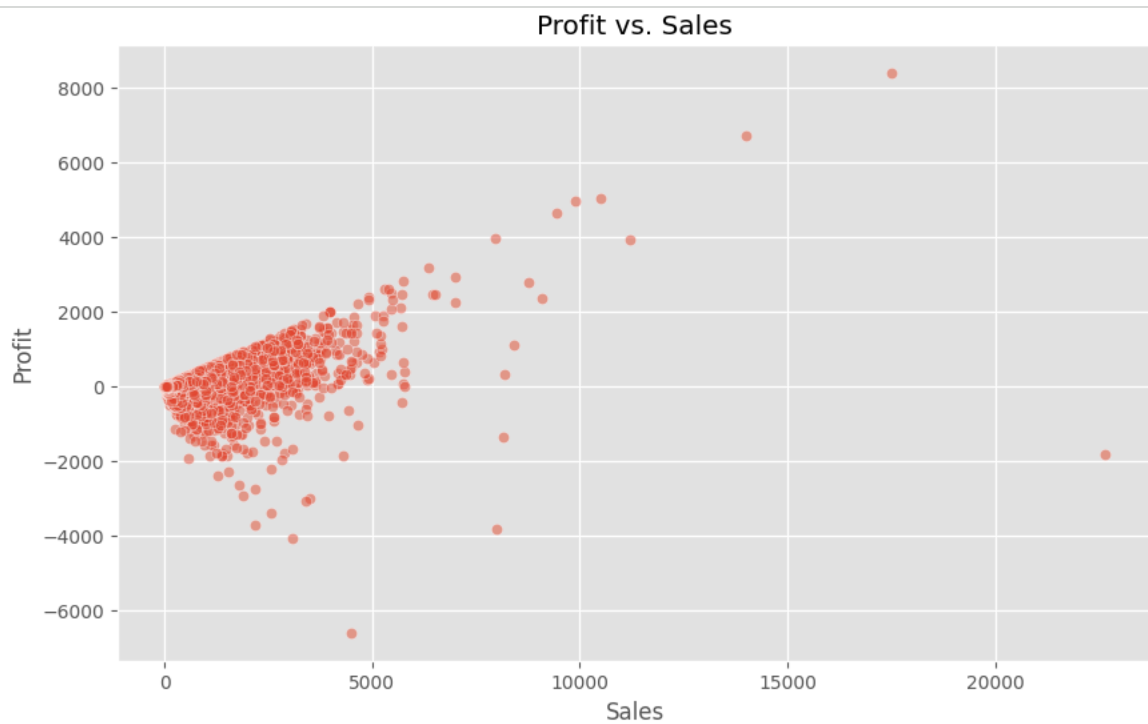

Distribution of Sales
Frequency
Sales Amount
0
10000
20000
30000
40000
50000
60000
70000
0
5000
10000
15000
20000

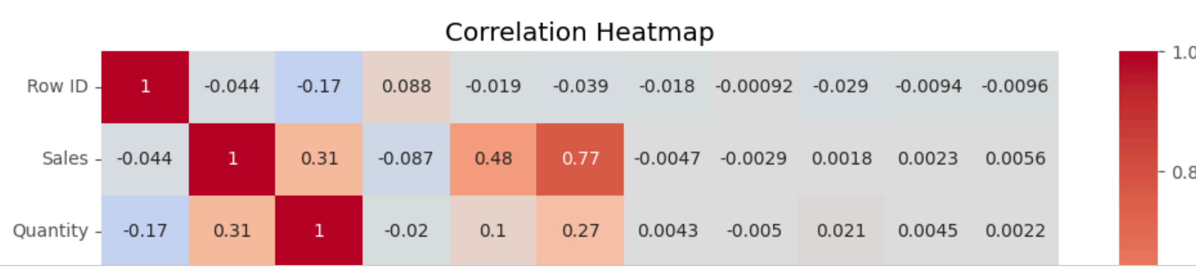

Total Sales by Product Category
1e6
Total Sales
Category
0
1
2
3
4
Technology
Furniture
Office Supplies


Total Sales by Region
1e6
Total Sales
1.0
1.5
2.0
2.5


0.5
0.0
East
Oceania
Central
Africa
West
South
Central Asia
EMEA
North Asia
North
Caribbean
Southeast Asia
Canada
Region


Monthly Sales Trend
Total Sales
500000
400000
300000
200000
100000
Month
2011-1
2011-2
2011-3
2011-4
2011-5
2011-6
2011-7
2011-8
2011-9
2011-10
2011-11
2011-12
2012-1
2012-2
2012-3
2012-4
2012-5
2012-6
2012-7
2012-8
2012-9
2012-10
2012-11
2012-12
2013-1
2013-2
2013-3
2013-4
2013-5
2013-6
2013-7
2013-8
2013-9
2013-10
2013-11
2013-12
2014-1
2014-2
2014-3
2014-4
2014-5
2014-6
2014-7
2014-8
2014-9
2014-10
2014-11
2014-12


Profit vs. Sales
Profit
8000
6000
4000
2000
0
−2000
−4000
−6000
0
5000
10000
15000
20000
Sales


Correlation Heatmap
Row ID
1 -0.044 -0.17 0.088 -0.019 -0.039 -0.018 -0.00092 -0.029 -0.0094 -0.0096
Sales
-0.044 1 0.31 -0.087 0.48 0.77 -0.0047 -0.0029 0.0018 0.0023 0.0056
Quantity
-0.17 0.31 1 -0.02 0.1 0.27 0.0043 -0.005 0.021 0.0045 0.0022
1.0
0.8

```
cat_cols = ['Ship Mode', 'Segment', 'City', 'State', 'Country', 'Market',
            'Region', 'Category', 'Sub-Category', 'Order Priority']

label_encoders = {}
for col in cat_cols:
    le = LabelEncoder()
    df[col] = le.fit_transform(df[col].astype(str))
    label_encoders[col] = le

features = ['Ship Mode', 'Segment', 'Category', 'Sub-Category', 'Quantity',
            'Discount', 'Shipping Cost', 'Order Priority', 'Processing Time',
            'Order Year', 'Order Month', 'Order Day', 'Order Weekday']

target = 'Sales'

X = df[features]
y = df[target]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

imputer = SimpleImputer(strategy='median')
X_train = imputer.fit_transform(X_train)
X_test = imputer.transform(X_test)
```

```
rf_model = RandomForestRegressor(n_estimators=100, random_state=42)
rf_model.fit(X_train, y_train)

y_pred = rf_model.predict(X_test)


mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print(f"Model Performance:")
print(f"RMSE: {rmse:.2f}")
print(f"R-squared: {r2:.2f}")

feature_importance = pd.DataFrame({
    'Feature': features,
    'Importance': rf_model.feature_importances_
}).sort_values('Importance', ascending=False)

plt.figure(figsize=(10, 6))
sns.barplot(x='Importance', y='Feature', data=feature_importance)
plt.title('Feature Importance')
plt.show()
```